Infraestructura de Servicios

PROF. ROQUE TRUJILLO RAMOS

Práctica de Kubernetes

UNIDAD 2

Ing.en Sistemas Computacionales

Realizado por:
Talavera Uribe Jose Miguel

Control del informe

TSkull-Shirt · Tienda de ropa alternativa con reservas por talla. Autor: Talavera Uribe Jose Miguel. Periodo documentado: 19 al 31 de octubre de 2026. Versión del informe: 1.0, corte del 31/10/2026. Producción de laboratorio, con cuentas y datos ficticios; no procesa pagos bancarios reales.

Repositorio: https://github.com/MakiMike/solucionestai_prueba.

D01. Ficha del proyecto y requerimientos

1.1 Problema, usuarios y alcance

Desarrollé una tienda de ropa alternativa para controlar inventario por prenda y talla. El problema principal es evitar sobreventas cuando varios clientes reservan las últimas unidades y recuperar el stock de pedidos que no se pagan. El cliente consulta catálogo, carrito, pedidos e historial; el administrador gestiona productos, tallas y usuarios, prepara pedidos y registra entregas. Se incluyen búsquedas, paginación, bajas lógicas y reporte de ventas. Se excluyen facturación fiscal, paqueterías y pagos financieros reales.

Reglas propias: RN01 el carrito no reserva; crear un pedido descuenta stock en una transacción. RN02 el plazo de pago es de 30 minutos; cancelar o vencer devuelve las unidades una sola vez. RN03 una edición limitada admite como máximo dos unidades por cliente, sumando tallas y pedidos vigentes o entregados. RN04 los precios se toman del catálogo y los reintentos no duplican pedidos ni pagos.

Flujo: Pendiente de pago → Confirmado → En preparación → Listo para entrega → Entregado. Desde Pendiente de pago también se permite Cancelado o Vencido. El cliente paga mediante simulación o cancela su propia reserva; el sistema vence pedidos y el administrador avanza las etapas de entrega. Se rechazan saltos de estado.

1.2 Criterios de aceptación y alcance comprobado

RF01 interfaz: login, catálogo, carrito y pedidos navegables. RF02 negocio: el navegador no accede a las bases; el servidor valida stock, precios y cantidades. RF03 CRUD: usuarios, productos y pedidos relacionados, con bajas lógicas o cancelación según entidad. RF04 roles: cliente sin acceso a administración ni pedidos ajenos. RF05 consultas: búsqueda, paginación y reporte de ventas. RF06 datos: 100 prendas, cinco tallas y 500 variantes; cuentas ficticias. RF07 historial: reservas, pagos y estados visibles desde MongoDB mediante outbox.

Pruebas concretas: pedido 2 entregado; producto 090 limitado rechazó tres unidades y una compra adicional en otra talla; al cancelar el pedido 6 el stock regresó a 20. El pedido 5 venció tras adelantar deliberadamente su fecha de expiración, y liberó stock sin duplicarlo. La autorización por URL rechazó un pedido ajeno existente. La revisión visual se hizo en escritorio/panel y el usuario confirmó acceso desde teléfono. No se acredita una auditoría exhaustiva de todos los formularios, sesión expirada, reporte de ventas y CRUD completo por API.

Objetivos no funcionales: dos réplicas mínimas por capa en producción; HPA de 2–6; p95 de catálogo ≤800 ms y errores <1 % para la carga medida; RPO objetivo 24 h y RTO objetivo 30 min. La meta original de 50 usuarios no se ensayó: la medición disponible usa 12. Se distingue objetivo de resultado en D09 y D10.

D02. Investigación y decisiones de arquitectura

Kubernetes mantiene el estado deseado mediante API server, etcd, scheduler y controladores; kubelet y containerd ejecutan las cargas en cada nodo. Un Pod es la unidad de ejecución; Deployment reemplaza réplicas y permite rolling update. Service y CoreDNS dan destinos estables: frontend:8080 envía FastCGI a backend:9000; el backend consulta postgres:5432 y mongo:27017. Ingress necesita su controlador; declararlo por sí solo no publica una aplicación.

Calico aporta red de Pods y cumplimiento de NetworkPolicy. CSI conecta las cargas con almacenamiento; PVC solicita capacidad y StatefulSet conserva identidad y asociación de volúmenes. ConfigMap guarda opciones y Secret guarda credenciales con RBAC; base64 no es cifrado. Readiness retira destinos no preparados, liveness detecta bloqueo del proceso y startup permite su inicio. HPA modifica Pods según métricas y requests; no agrega máquinas. La distribución por hostname y PDB limitan efectos de mantenimiento, sin eliminar puntos únicos de fallo.

2.1 Comparación y decisiones registradas

ADR01 Distribución: elegí Minikube 1.39.0 con driver Docker frente a kind. Minikube facilita perfiles independientes y addons; kind era la propuesta inicial y sus archivos quedaron como material histórico. Ambos consumen el mismo host. Costo: memoria, disco y dependencia de Docker Desktop/WSL 2; no hay aislamiento físico.

ADR02 Red: Calico 3.32.2 frente a una red sin motor de políticas. Permite demostrar denegación y permisos explícitos. Costo: agentes por nodo y mantenimiento de credenciales CNI. No sustituye al firewall de Windows ni controla todo hostNetwork.

ADR03 Entrada: Traefik 3.7.16, chart 42.1.3, con Ingress frente a Envoy Gateway/Gateway API. Conserva las rutas existentes; para el alias IP usa IngressRoute y TLSStore. Se publica mediante port-forward, no mediante una VIP de nube. Costo: dos Pods de entrada y certificados; depende de un proceso en Windows y de la API.

ADR04 Persistencia: CSI hostpath para las bases y NFS CSI 4.13.0 para uploads compartido, frente a archivos efímeros por contenedor. Evita catálogos distintos entre réplicas. Costo: disco y servidor NFS; los volúmenes locales y el NFS único no ofrecen failover.

ADR05 Datos: PostgreSQL como fuente de verdad y MongoDB como historial documental, frente a suponer una transacción común entre motores. Outbox guarda el evento con la compra y un envío idempotente lo replica después. Costo: dos motores y consistencia eventual; el historial puede retrasarse.

ADR06 Observabilidad: kube-prometheus-stack 92.13.1 frente a depender solo de kubectl top/logs. Prometheus conserva métricas, Grafana presenta paneles y Alertmanager administra alertas. Exporters consultan ambos motores con usuarios propios. Costo: CPU, RAM y PVC; la centralización de logs propuesta con Loki/Alloy no se implementó.

D03. Arquitecturas, inventario y capacidad

1. Arquitectura de la aplicación

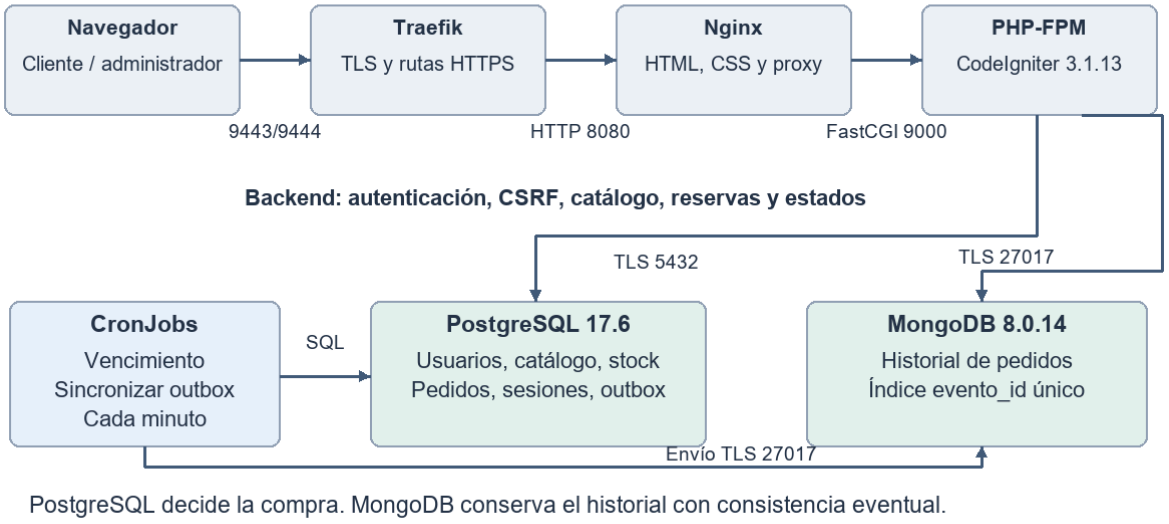


Figura 1. Responsabilidades y protocolos de TSkull-Shirt. En producción los dos motores usan TLS; la interfaz se renderiza en el servidor.

2. Secuencia CRUD: modificar una prenda

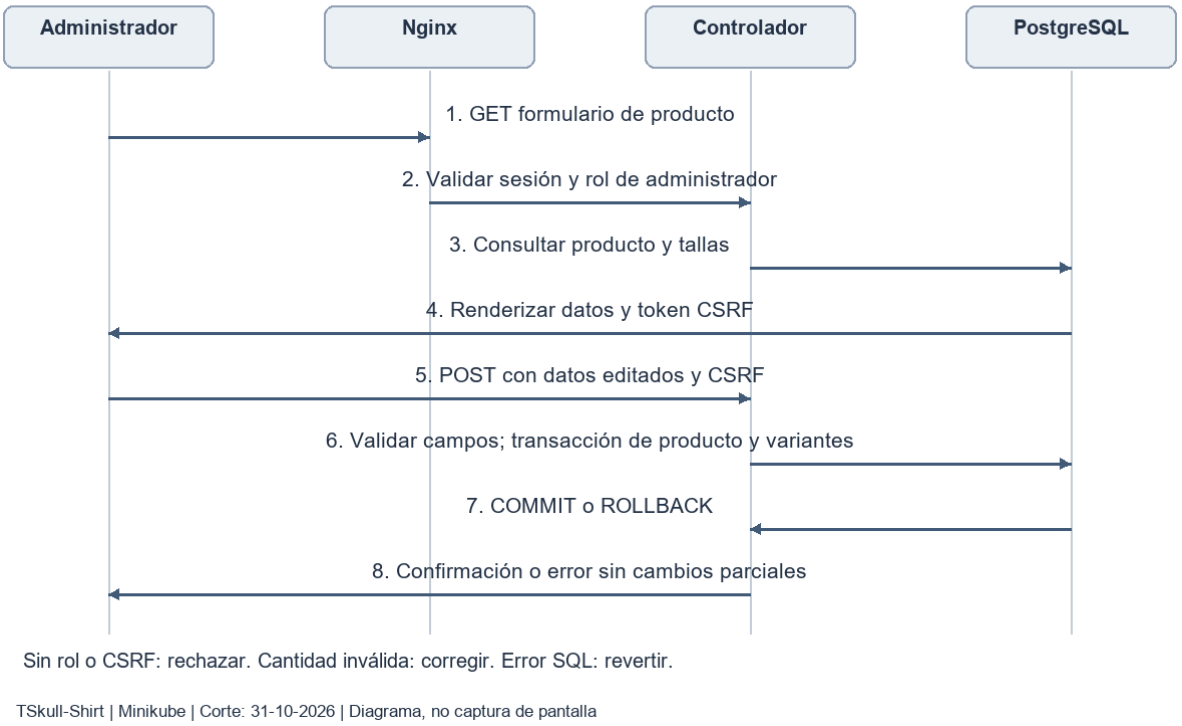


Figura 2. Secuencia CRUD de producto: autorización, validación, escritura y reversión ante error.

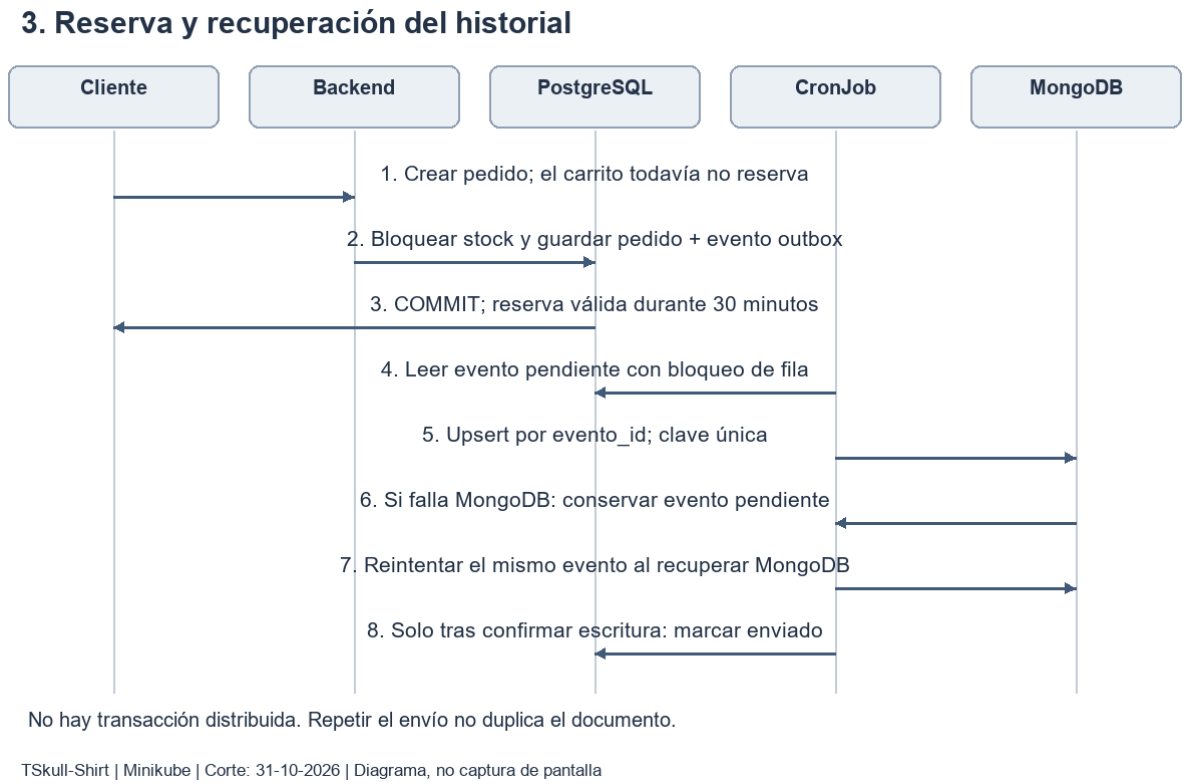


Figura 3. Compra y envío del historial: PostgreSQL conserva el evento si MongoDB falla; el índice único evita duplicados al reintentar.

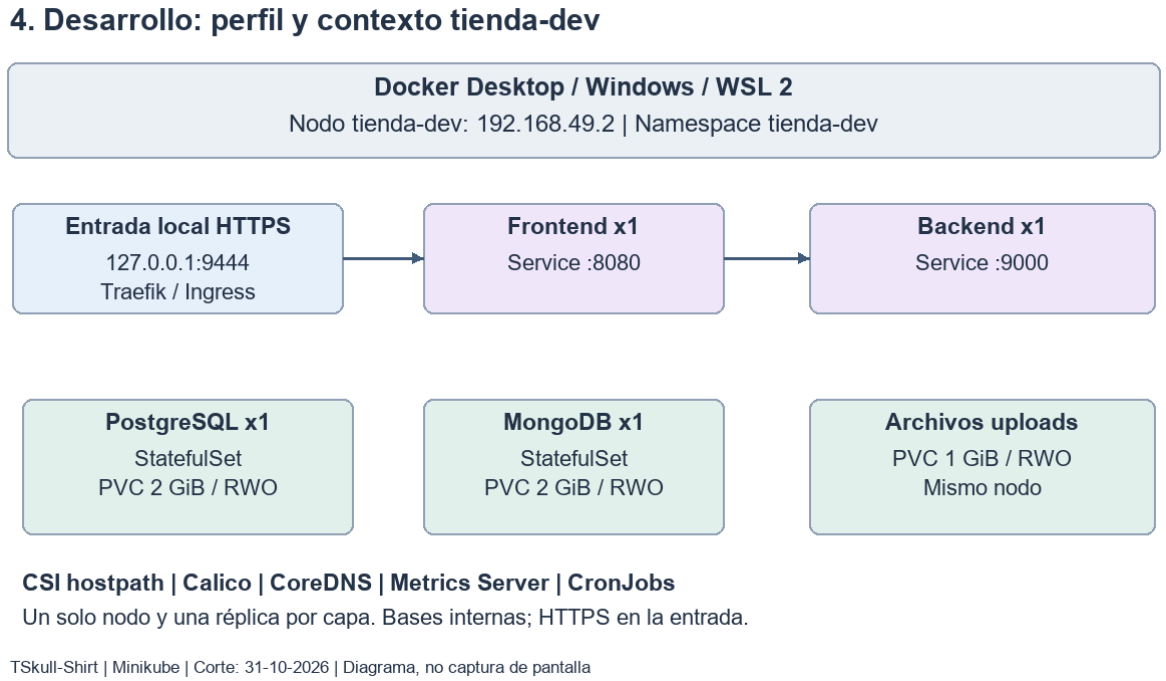


Figura 4. Desarrollo: un nodo y una réplica inicial por capa, con datos y credenciales propios.

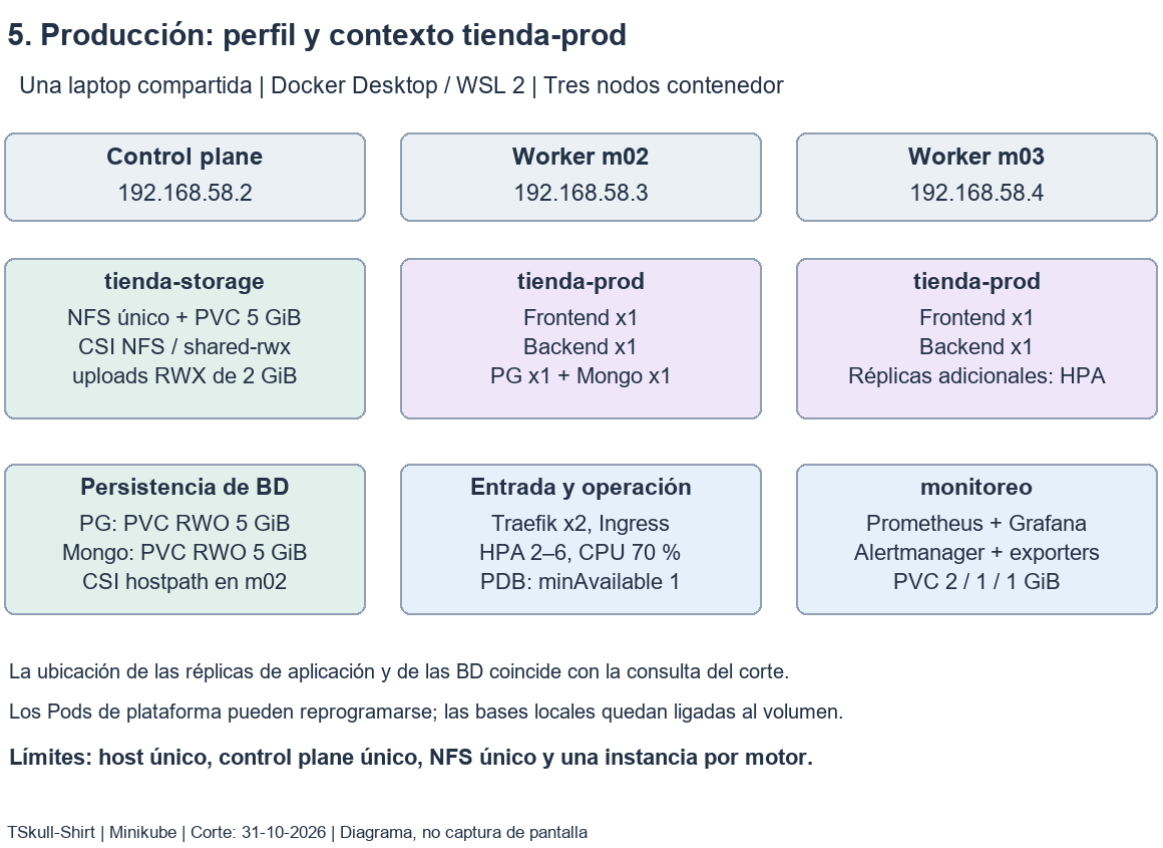


Figura 5. Producción: control plane y dos workers; réplicas de aplicación distribuidas y almacenamiento no redundante.

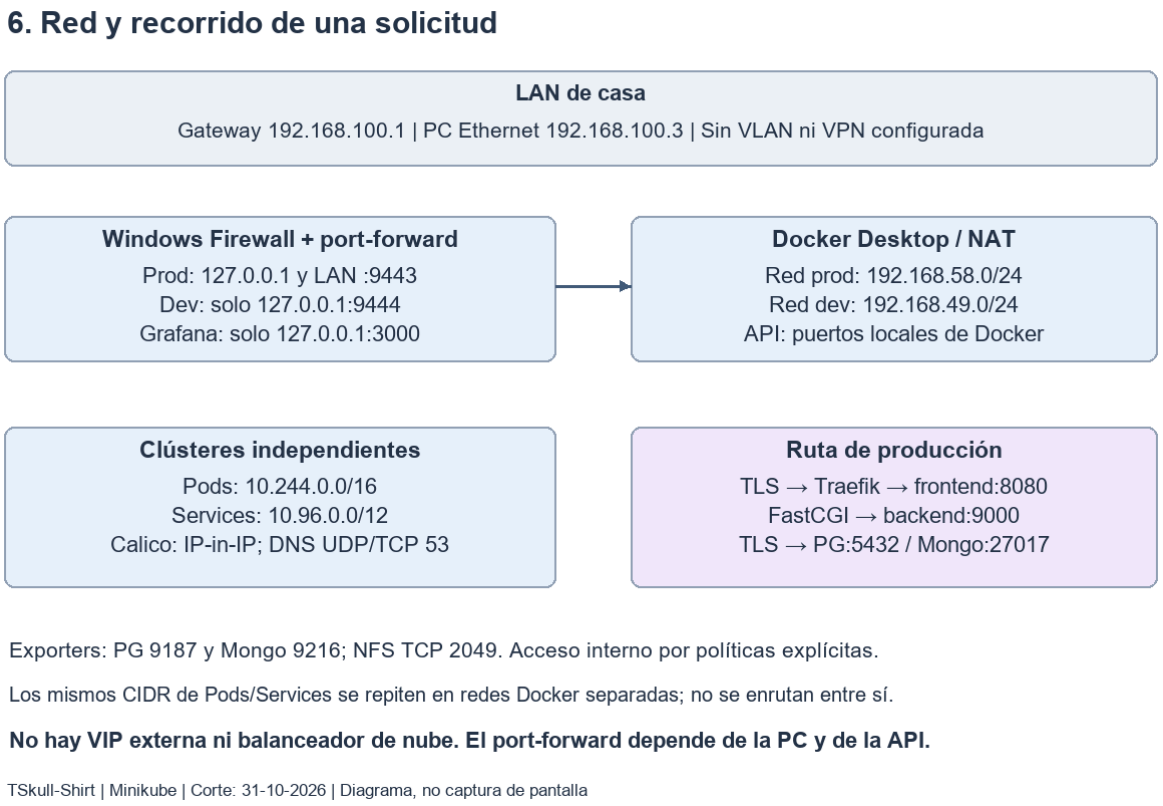


Figura 6. Recorrido desde LAN o localhost. Los rangos internos repetidos pertenecen a redes Docker independientes; no existe una VIP externa.

3.1 Inventario y presupuesto real

Equipo físico: Windows 11 Home 25H2, Intel Core i5-12450H, 8 núcleos/12 hilos, 23.8 GiB de RAM; unidad C: 474.7 GiB, con 5.7 GiB libres al corte. Arquitectura amd64. Docker Desktop 29.0.1 usa WSL 2. Cada contenedor-nodo tiene límite Docker de 2 CPU y 3 GiB; cuatro nodos suman un presupuesto máximo de 8 CPU y 12 GiB, además del host y herramientas. Kubernetes informa capacidad de la VM compartida: no se sumó como si cada nodo tuviera hardware independiente.

Desarrollo: tienda-dev, 192.168.49.2. Producción: tienda-prod, 192.168.58.2; tienda-prod-m02, 192.168.58.3; tienda-prod-m03, 192.168.58.4. El acceso LAN de casa usa Ethernet 192.168.100.3 y gateway 192.168.100.1. Las IP físicas y los puertos Docker pueden cambiar. No se configuró VLAN ni VPN. Calico usa IP-in-IP (protocolo IP 4), no VXLAN, en el pool observado.

La captura histórica de kubectl top mostró backend alrededor de 10–11 mCPU y 18–19 MiB; frontend 1 mCPU y 10–17 MiB; MongoDB 183 MiB y PostgreSQL 37 MiB. Son muestras puntuales, no máximos de capacidad. El disco libre es un riesgo actual. HPA no garantiza que quepan seis Pods: también limitan requests, cuotas y recursos del host. La pérdida de esta laptop detiene los dos ambientes.

```
PS C:\laragon\www\sistema> minikube -p tienda-dev kubectl -- logs job/verificar-bases -n tienda-dev
Usuario PostgreSQL: tienda_app
playeras: 100
tallas: 5
playera_tallas: 500
us_usuarios: 2
MongoDB: acceso a eventos con usuario de aplicacion OK
Indice: _id_
Indice: evento_unico
Indice: pedido_fecha
VERIFICACION COMPLETA
PS C:\laragon\www\sistema> minikube -p tienda-dev kubectl -- get deployments,statefulsets,pvc -n tienda-dev
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend      1/1     1            1           97s
deployment.apps/frontend     1/1     1            1           97s

NAME                          READY   AGE
statefulset.apps/mongo        1/1     5m53s
statefulset.apps/postgres     1/1     5m53s

NAME                                      STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS      VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-mongo-0        Bound    pvc-98f2ea57-72e3-42c4-8944-b8cb24229860   2Gi        RWO            csi-hostpath-sc   <unset>                 5m53s
persistentvolumeclaim/data-postgres-0     Bound    pvc-1f95a079-90d6-4a43-acbd-0319e929ca6e   2Gi        RWO            csi-hostpath-sc   <unset>                 5m53s
persistentvolumeclaim/uploads             Bound    pvc-0dbaec31-f96d-4986-8304-a206b91013ac   1Gi        RWO            csi-hostpath-sc   <unset>                 6m23s
PS C:\laragon\www\sistema>
```

Captura 1 · E01/E04 · 26–27/10/2026, desarrollo. Verificación de ambas bases, 100 prendas, cinco tallas, 500 variantes y cargas/PVC disponibles. Es un registro del levantamiento; las versiones finales se indican en D05.

D04. Modelos de datos y consistencia

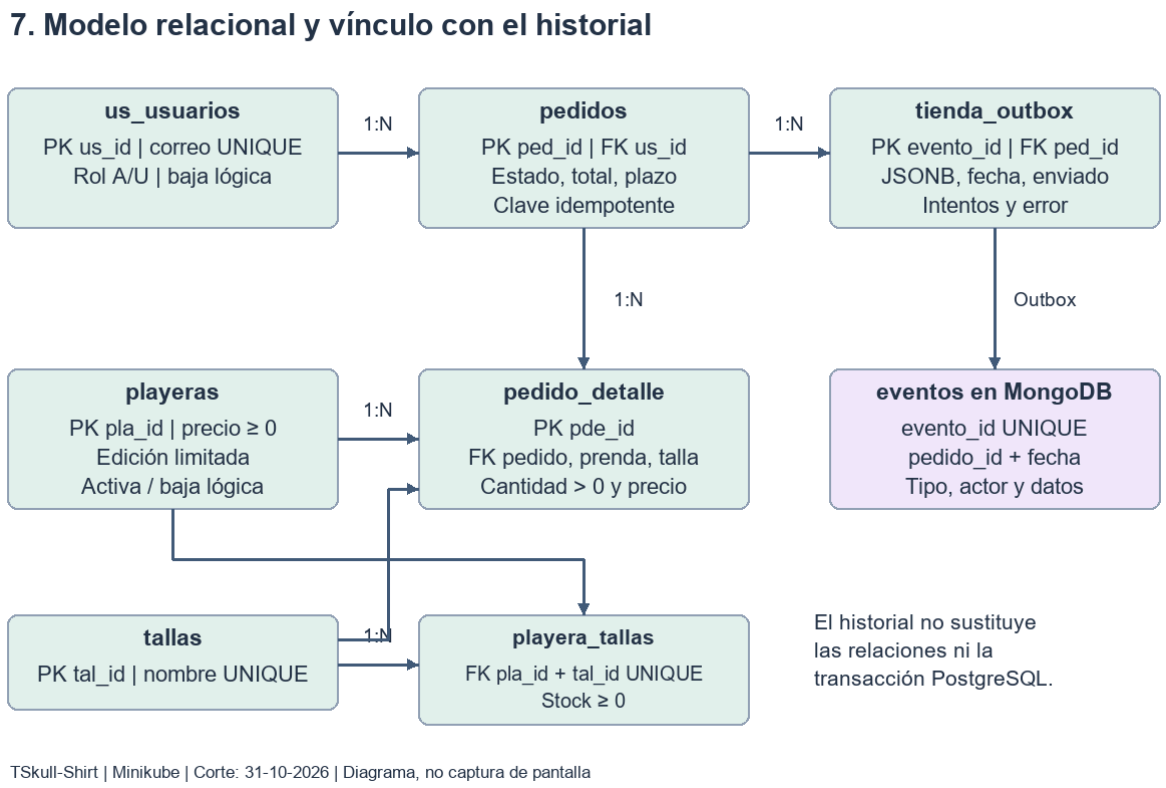


Figura 7. Relaciones principales de PostgreSQL y vínculo lógico con eventos en MongoDB. ci_sessions almacena las sesiones compartidas.

PostgreSQL 17.6-bookworm: us_usuarios usa PK us_id y correo único; tallas usa PK tal_id y nombre único; playeras usa PK pla_id, precio numeric(12,2) no negativo y baja lógica; playera_tallas relaciona prenda/talla con combinación única y stock entero no negativo. pedidos referencia usuario, estado, total, vencimiento y clave idempotente por cliente. pedido_detalle relaciona pedido/prenda/talla, cantidad positiva y precio/nombre/talla congelados al comprar. tienda_outbox contiene evento_id, pedido, actor, tipo, datos JSONB, fecha, envío e intentos.

Índices: pedidos por usuario/estado, vencimiento de pendientes, detalles por pedido, variante única y outbox no enviado. Las transacciones bloquean variantes antes de reservar y comprueban nuevamente existencias y límite acumulado. Las conexiones se abren por proceso PHP con tiempos de espera; no se instaló PgBouncer. El máximo conjunto debe presupuestar réplicas, procesos FPM, CronJobs, migración y monitorización; no se midió saturación de conexiones.

```
SELECT ped_estado, count(*) FROM pedidos GROUP BY ped_estado;
SELECT pla_id,tal_id,pta_stock FROM playera_tallas WHERE pla_id=98;
SELECT evento_id,ped_id,tipo FROM tienda_outbox WHERE enviado_en IS NULL;
```

Migraciones: database/migrations/001_base.sql, 002_reservas_outbox.sql, 003_usuario_secuencia.sql y 004_bajas_logicas.sql; ejecución única con bin/tienda.php migrate. La semilla idempotente genera 100 prendas, cinco tallas y 500 variantes sin restablecer existencias modificadas. No se ejecuta una migración por cada réplica.

MongoDB 8.0.14: colección eventos. Campos obligatorios: evento_id y pedido_id enteros, tipo string, fecha BSON Date y datos objeto; actor_id admite identidad o null. Un validador JSON Schema comprueba tipos. Índices _id_, evento_unico (evento_id único) y pedido_fecha (pedido_id ascendente, fecha descendente). Consulta representativa: db.eventos.find({pedido_id:2}).sort({fecha:-1}). No es una caché: alimenta la línea de tiempo visible.

Outbox: el cambio y el evento se confirman juntos en PostgreSQL; sync usa FOR UPDATE SKIP LOCKED, upsert con $setOnInsert y writeConcern majority. Solo marca enviado tras confirmación. Al fallar MongoDB conserva el pendiente e incrementa intentos. El error inicial con el driver MongoDB 2.1.0 se corrigió pasando writeConcern dentro del arreglo de opciones; el Job historial-compat-014 sincronizó dos eventos. El reintento sin duplicación está cubierto por tests/history-driver.php; no se acredita un ensayo integral nuevo apagando el motor y repitiendo toda la compra.



Captura 2 · E03/E04 · 27/10/2026, desarrollo. Pedido entregado e historial sincronizado consultable. Los números de pedido visibles pertenecen a la versión anterior; actualmente el cliente no los ve.

D05. Manual de instalación y reanudación

Versiones al corte: Kubernetes 1.37.0, Minikube 1.39.0, containerd 2.3.4, Calico 3.32.2; backend tienda-backend:0.1.6 con PHP 8.3.36, frontend tienda-frontend:0.1.1; Traefik chart 42.1.3/app 3.7.16; kube-prometheus-stack 92.13.1. El repositorio separa aplicación en application/, Dockerfile en docker/php/, pruebas en tests/, producción en k8s/minikube/ y desarrollo en su subcarpeta desarrollo/. Las carpetas kind y overlays previas no son el procedimiento final.

Estos pasos describen la instalación por fases; su reconstrucción completa en recursos limpios no fue ensayada. No aplicarlos indiscriminadamente sobre los ambientes existentes. Los anexos amplían comandos, prerrequisitos, comprobaciones y problemas. Se excluyen de Git k8s/tls/, k8s/secrets.env, .env, output/, tmp/ y vendor/. No se publican valores de credenciales.

5.1 Crear la plataforma, solamente en recursos nuevos

Paso 1. Preparar Windows, virtualización, Docker Desktop/WSL 2, Git, Minikube, Helm y mkcert. Abrir Docker, comprobar docker info y resolución de nombres. Construir y probar imágenes con dependencias fijadas. Si Docker no responde, iniciar su motor antes de Minikube. Evidencia: E01 y reportes de imágenes.

```
docker build --target backend -f docker/php/Dockerfile -t tienda-backend:0.1.6 .
docker build --target frontend -f docker/php/Dockerfile -t tienda-frontend:0.1.1 .
```

Paso 2. Crear perfiles independientes con Calico. Cada perfil genera su contexto y red; no sustituirlos por dos namespaces. Verificar get nodes. Ante NotReady, revisar componentes CNI y eventos; no borrar volúmenes para corregir un arranque.

```
minikube start -p tienda-dev --driver=docker --container-runtime=containerd --nodes=1
--cni=calico --cpus=2 --memory=3072 --kubernetes-version=v1.37.0
```

```
minikube start -p tienda-prod --driver=docker --container-runtime=containerd --nodes=3
--cni=calico --cpus=2 --memory=3072 --kubernetes-version=v1.37.0
```

Paso 3. En cada perfil habilitar metrics-server y csi-hostpath-driver; cargar las dos imágenes. En producción instalar CSI NFS desde k8s/minikube/csi-nfs-v4.13.0/, servidor-nfs.yaml y volumen-fotos.yaml. Comprobar Pods CSI y PVC Bound; la advertencia de volumesnapshots no demuestra fallo de persistencia y no se ensayaron snapshots.

```
minikube -p tienda-dev addons enable metrics-server
```

```
minikube -p tienda-dev addons enable csi-hostpath-driver
```

```
minikube -p tienda-dev image load tienda-backend:0.1.6 tienda-frontend:0.1.1
```

```
minikube -p tienda-prod addons enable metrics-server
```

```
minikube -p tienda-prod addons enable csi-hostpath-driver
```

```
minikube -p tienda-prod image load tienda-backend:0.1.6 tienda-frontend:0.1.1
```

5.2 Configuración, datos y publicación

Paso 4. Desarrollo: aplicar desarrollo/plataforma.yaml, ejecutar desarrollo/Preparar-Credenciales.ps1 y crear tienda-secrets desde su archivo privado. Producción: preparar namespace/ServiceAccount/ConfigMap, completar k8s/secrets.env.example en un archivo privado y crear su Secret; aplicar seguridad-red.yaml, permisos-operacion.yaml y cuotas-recursos.yaml. Prerrequisito: credenciales de ambos ambientes diferentes. Comprobar RBAC y rechazo de excesos. Nunca mostrar los archivos privados en la terminal de una captura.

```
minikube -p tienda-dev kubectl -- apply -f k8s/minikube/desarrollo/plataforma.yaml
```

```
powershell -NoProfile -ExecutionPolicy Bypass -File
k8s/minikube/desarrollo/Preparar-Credenciales.ps1
```

```
minikube -p tienda-dev kubectl -- create secret generic tienda-secrets -n tienda-dev
--from-env-file=k8s/tls/dev-private/secrets.env
```

```
minikube -p tienda-prod kubectl -- create secret generic tienda-secrets -n tienda-prod
--from-env-file=k8s/secrets.env
```

Paso 5. Aplicar bases-datos.yaml del ambiente; esperar ambos StatefulSets. Aplicar inicializar-tienda.yaml una sola vez, esperar Job complete y revisar logs de migraciones/semilla. En producción existe inicializar-mongo.yaml para preparar el motor. Después activar TLS con Habilitar-TlsPostgres.ps1 y Habilitar-TlsMongo.ps1 y ejecutar sus scripts Comprobar-Tls. La versión final de los manifiestos de aplicación exige las CA: en una instalación nueva el orden TLS debe completarse antes de publicar. El proceso limpio completo sigue pendiente de validar.

```
minikube -p tienda-dev kubectl -- apply -f k8s/minikube/desarrollo/bases-datos.yaml
```

```
minikube -p tienda-dev kubectl -- apply -f k8s/minikube/desarrollo/inicializar-tienda.yaml
```

```
minikube -p tienda-dev kubectl -- wait --for=condition=complete job/tienda-migrate -n
tienda-dev --timeout=300s
```

```
minikube -p tienda-prod kubectl -- apply -f k8s/minikube/bases-datos.yaml -f
k8s/minikube/inicializar-tienda.yaml
```

Paso 6. Aplicar aplicacion.yaml y tareas-programadas.yaml propios del ambiente. Instalar Traefik chart 42.1.3 con los valores de cada perfil; preparar nombres locales, certificados

mkcert, Secret tienda-tls e Ingress entrada-https.yaml. Verificar rollout y HTTPS. Frontend sirve estáticos y delega PHP por FastCGI. Un error 404 al abrir una IP no demuestra bloqueo de firewall: puede faltar una ruta Host.

```
minikube -p tienda-dev kubectl -- apply -f k8s/minikube/desarrollo/aplicacion.yaml -f
k8s/minikube/desarrollo/tareas-programadas.yaml
```

```
minikube -p tienda-prod kubectl -- apply -f k8s/minikube/aplicacion.yaml -f
k8s/minikube/tareas-programadas.yaml
```

```
helm upgrade --install traefik traefik/traefik --version 42.1.3 --kube-context tienda-prod
-n kube-system -f k8s/minikube/traefik-values.yaml --wait
```

Paso 7. Instalar monitoreo, preparar usuarios de exporters y aplicar metricas-postgres.yaml, metricas-mongo.yaml y alertas-bases.yaml. Programar respaldos a las 20:00. Paso 8. Validar carga, fallos, permisos y restauración aislada según D06–D10. No confundir una actualización correcta con reconstrucción desde cero.

```
helm upgrade --install monitoreo prometheus-community/kube-prometheus-stack --version
92.13.1 --kube-context tienda-prod -n monitoreo --create-namespace -f
k8s/minikube/monitoreo-values.yaml --wait --timeout 10m
```

5.3 Encender y apagar los ambientes existentes

Abrir Docker Desktop. Iniciar producción conservando el parámetro de cifrado de Secrets y luego desarrollo. Dejar un port-forward por terminal. Las IP LAN deben comprobarse cada vez; la prueba actual de casa usa 192.168.100.3. Al terminar, Ctrl+C en esas terminales, detener ambos perfiles y cerrar Docker Desktop. No usar minikube delete.

```
minikube start -p tienda-prod
--extra-config=apiserver.encryption-provider-config=/var/lib/minikube/certs/encryption-conf
ig.json
```

```
minikube start -p tienda-dev
```

```
minikube -p tienda-prod kubectl -- port-forward -n kube-system service/traefik 9443:443
--address=127.0.0.1,192.168.100.3
```

```
minikube -p tienda-dev kubectl -- port-forward -n kube-system service/traefik 9444:443
--address=127.0.0.1
```

```
minikube -p tienda-prod kubectl -- port-forward -n monitoreo service/monitoreo-grafana
3000:80 --address=127.0.0.1
```

```
minikube stop -p tienda-dev
```

```
minikube stop -p tienda-prod
```

D06. Seguridad por capas y matriz de flujos

Flujos permitidos: usuario local → HTTPS prod 9443/dev 9444; LAN autorizada → prod 9443; Traefik → frontend TCP 8080; frontend → backend TCP 9000; backend/CronJobs → PostgreSQL 5432 y MongoDB 27017; Pods autorizados → DNS UDP/TCP 53; Prometheus → exporters PG 9187/Mongo 9216; cargas de archivos → NFS TCP 2049. El control entre nodos incluye API/kubelet y Calico IP-in-IP/BGP según configuración, dentro de las redes Docker; no se abrieron indiscriminadamente en la LAN.

Calico aplica denegación inicial de entrada/salida y excepciones por selectores. Comprobar-Red.ps1 verificó backend hacia ambas bases; el frontend resolvió mongo por DNS y su conexión a 27017 agotó el tiempo de espera contra el mismo servicio sano. No se aporta una medición separada completa de todos los sentidos ingress/egress. Las bases no se publican por Ingress ni NodePort.

Windows Firewall conserva perfiles activos y bloqueo entrante; una regla Block protege desarrollo 9444, Grafana 3000, Prometheus 9090, restauración 9081 y puertos Docker del laboratorio. La regla Allow de producción limita 9443 a LocalSubnet, interfaz Ethernet e IP 192.168.100.3; 9443 se retira del Block porque Block prevalece. El 31/10/2026 el usuario confirmó desde su teléfono que prod abrió y dev no. Dev además escucha solo en loopback: esta prueba valida la restricción efectiva, no atribuye por sí sola el resultado exclusivamente al firewall. Exportaciones: output/evidencias/seguridad/.

Aplicación: hash de contraseñas, autorización en servidor, CSRF y cookies Secure/HttpOnly. Consultas de compra parametrizadas; búsqueda del catálogo corregida para escapar comillas y comodines. No se acredita una prueba completa de límites de intentos ni auditoría de todos los endpoints. Datos: usuarios distintos de migración, aplicación y monitoreo. Secretos y kubeconfig no se entregan como anexos.

RBAC: tienda-observador consulta Pods pero no lee Secrets ni escala; tienda-operador consulta y modifica exclusivamente el subrecurso scale de backend/frontend. No puede editar el Deployment completo, StatefulSets, exec, RoleBindings, Secrets ni el namespace monitoreo. Las 12 respuestas coincidieron con lo esperado; consultar Secrets como observador produjo Forbidden. PSA Restricted se aplica a cargas propias, con usuario no root, capabilities reducidas, sin escalamiento y seccomp RuntimeDefault. El namespace de almacenamiento admite la excepción necesaria del servidor NFS.

TLS de datos: PostgreSQL usa verify-full y su CA; MongoDB usa TLS con CA, hostname y autenticación. Se verificó TLS 1.3/AES-256-GCM; nombre incorrecto y TCP sin TLS fueron rechazados. El acceso móvil usa certificado para la IP, TLSStore predeterminado y la CA pública instalada por el usuario. No se desactivó la validación TLS.

Secrets en reposo: secretbox con clave aleatoria de 256 bits en archivo root:600 del control plane. Se reescribieron y comprobaron 24 Secrets en etcd; la API pudo leerlos. La copia local de la clave usa DPAPI del usuario de Windows. No se eliminaron versiones históricas ni existe KMS o rotación automática. El primer copiado a /tmp falló por permisos; se transfirió a un directorio de etapa y se instaló con modo 600. Conservar la clave y el parámetro del API server al reiniciar.

6.1 Hallazgos y riesgos residuales

Docker Scout: backend 0.1.2 pasó de 201 vulnerabilidades a 195 en 0.1.3; frontend 0.1.0 pasó de 191 a 31 en 0.1.1. Son conteos de esos escaneos, no un escaneo nuevo del backend 0.1.6. Persisten hallazgos críticos y altos; no se declara el sistema libre de vulnerabilidades. Se actualizó PHP a 8.3.36 y paquetes del sistema; se deben revisar dependencia, aplicabilidad y nueva base, evitando descartar todas las alertas de CI3 como falsos positivos.

Responsable de corrección: autor del proyecto. Prioridades abiertas: críticas/altas de imágenes; poco disco libre; dependencia de host/NFS/motores únicos; recuperación DPAPI ligada al perfil de Windows; logs no centralizados; trazabilidad final de 0.1.6 sin commit/tag. Las cuentas son ficticias y no se publica la contraseña. No se recomienda usar este laboratorio como tienda comercial sin resolver estos riesgos.

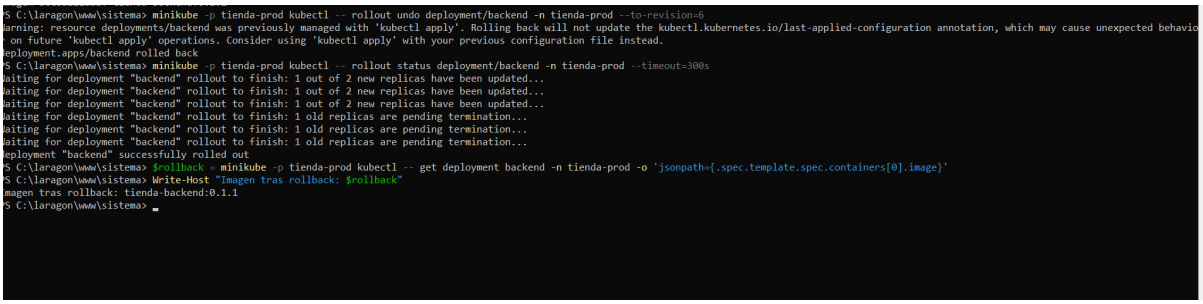
D07. Publicación, promoción y cambios

Desarrollo: https://tienda-dev.test:9444/, contexto/namespace tienda-dev, una réplica por capa. Producción: https://tskull-shirt.test:9443/, contexto/namespace tienda-prod, dos réplicas mínimas; tienda-prod.test se conserva como alias. Grafana: http://127.0.0.1:3000/. En casa, el teléfono usa https://192.168.100.3:9443/ después de confiar en la CA pública. Cambiar de red requiere actualizar IP, certificado, ruta, alias y firewall; no hay publicación automática permanente.

Promoción: construir una etiqueta nueva, probar el mismo artefacto en dev, cargarlo en todos los nodos de prod, revisar dry-run y aplicar manifiestos. Verificar imagen, réplicas, HTTPS y funcionamiento. Git registra el código y la configuración; no despliega por sí solo. No se configuró CI/CD ni una etiqueta final de entrega. Los certificados, Secret y datos se conservan por ambiente.

```
minikube -p tienda-prod image load tienda-backend:0.1.6
minikube -p tienda-prod kubectl -- apply --dry-run=server -f k8s/minikube/aplicacion.yaml -f k8s/minikube/tareas-programadas.yaml
minikube -p tienda-prod kubectl -- apply -f k8s/minikube/aplicacion.yaml -f k8s/minikube/tareas-programadas.yaml
minikube -p tienda-prod kubectl -- rollout status deployment/backend -n tienda-prod --timeout=300s
```

Ensayo registrado: backend 0.1.1 → 0.1.2, revisión previa 6; rollout undo --to-revision=6 devolvió 0.1.1 y después se promovió nuevamente 0.1.2 con HTTP 200 y 2/2. El aviso de last-applied durante undo se conservó; los manifiestos se alinearon después. Estado actual: backend 0.1.6 y frontend 0.1.1. Un rollback de imagen no revierte esquema ni datos; no se aplicó una migración incompatible en este ensayo.



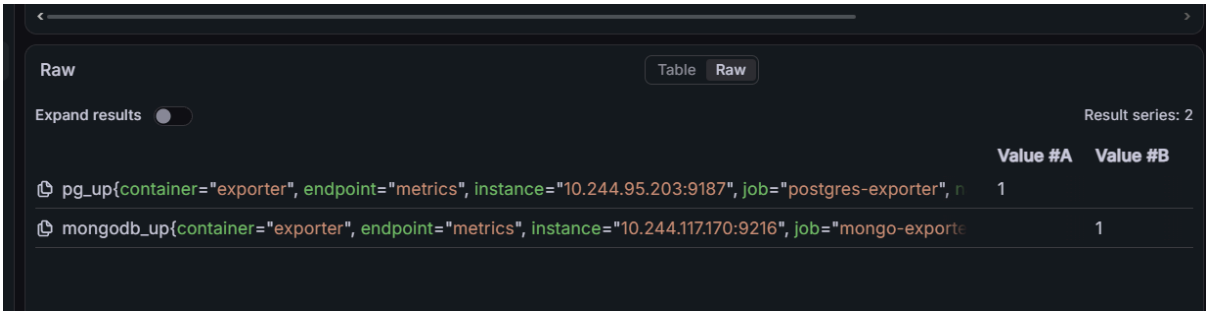
Captura 3 · E12 · 26/10/2026, producción. Rollback a la revisión 6: rollout satisfactorio e imagen 0.1.1. Demuestra la reversión histórica, no la versión actual.

Las sesiones se guardan en PostgreSQL y uploads es compartido por NFS en producción. El cliente ya no ve el ID global en el encabezado del pedido; el administrador conserva números y búsqueda. La autorización sigue siendo por propietario/rol; ocultar el número no es el control de seguridad. Alta/cambio/revocación de operadores se realiza mediante RoleBindings y comprobaciones auth can-i; no se comparte el kubeconfig administrador.

D08. Operación, observabilidad e incidentes

La pila monitoreo incluye Prometheus, Grafana, Alertmanager, kube-state-metrics y node-exporter; retención de Prometheus 24 h, scrape/evaluación cada 30 s y PVC propios. Exporters de PostgreSQL y MongoDB se descubren mediante ServiceMonitor. pg_up=1 y

mongodb_up=1 acreditan verificación correcta desde sus exporters; un exporter Running por sí solo no lo demuestra. Metrics Server alimenta HPA/top y no sustituye Prometheus.



Captura 4 · E10 · 26/10/2026, producción. pg_up y mongodb_up en 1 después de la configuración TLS. Se seleccionó esta comprobación conjunta para evitar repetir paneles de cada motor.

Alertas ensayadas: TiendaPostgresSinVerificacion y TiendaMongoSinVerificacion, críticas tras un minuto sin disponibilidad verificada o sin serie. Se restringió temporalmente la salida de los exporters con prueba-alertas-bases.yaml; ambas aparecieron firing. Al restaurar metricas-postgres.yaml y metricas-mongo.yaml, los indicadores regresaron a 1 y las alertas dejaron de estar activas. Esto prueba falta de verificación desde monitoreo, no una caída real de los motores.

Respuesta: revisar primero red, exporter, credenciales y motor; no reiniciar la base solo por esta alerta. Las reglas incluidas por el chart complementan la vigilancia, pero no se demuestra una matriz propia completa de las cinco alertas exigidas (aplicación, nodo, errores, saturación y respaldo vencido). Tampoco se desplegó Loki/Alloy ni se validaron paneles de p95 continuo, logs centralizados y antigüedad del respaldo. Se registra como cumplimiento parcial de observabilidad.

8.1 Manual breve de incidentes

Pod backend: comprobar get pods, describe y logs; observar su recreación y la sonda HTTPS. Eliminar un único Pod durante la prueba produjo 81 respuestas 200 y cero fallos en 90.3 s; no se midieron cinco minutos. No borrar PVC. Si no recupera, revisar eventos, endpoints, volumen, recursos y dependencias antes de escalar.

Mantenimiento de worker: cordon m03, drain limitado al selector app in (frontend,backend), ignorando DaemonSets y respetando PDB, verificar ambos rollouts y finalizar con uncordon. Se reubicó la aplicación: 159 respuestas 200 y un 502 en 180.3 s. No fue un drain completo ni una pérdida abrupta de nodo; se mantuvieron fuera las cargas de datos. El error transitorio se conserva como resultado real.

Base o historial no disponible: revisar salud, DNS, NetworkPolicy, CA y usuarios; confirmar que el pedido permanezca en PostgreSQL y el evento pendiente en outbox. Recuperar la dependencia y ejecutar sync, comprobar enviados y eventos únicos. El fallo de compatibilidad del driver fue corregido y sincronizó dos eventos; la prueba completa de caída/reconciliación en versión final queda parcial.

Actualización fallida: detener la promoción, revisar describe/logs y volver a la revisión registrada; verificar imagen y HTTPS. Reanudar tras apagado: si CNI informa Unauthorized, diagnosticar Calico; en el arranque del corte se reinició solo daemonset/calico-node, se esperó el rollout y se recuperaron las cargas. Si se reinicia Traefik, abrir de nuevo sus port-forward.

D09. Persistencia, respaldos y continuidad

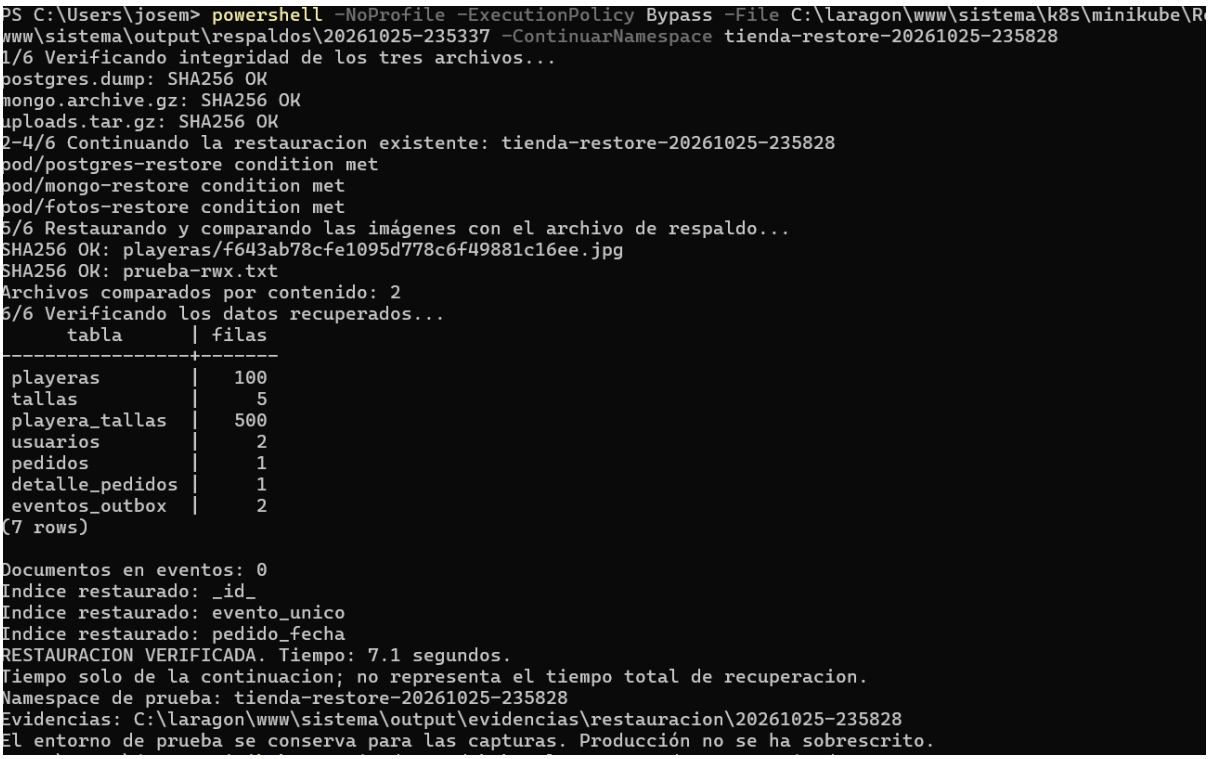
Producción: data-postgres-0 y data-mongo-0, 5 GiB RWO cada uno, csi-hostpath-sc; uploads, 2 GiB RWX, shared-rwx sobre NFS CSI. Servidor NFS único con PVC 5 GiB. Desarrollo: 2 GiB por motor y uploads 1 GiB RWO en el mismo nodo. Los PVC están Bound y separados por clúster. La clase CSI local tiene reclaimPolicy Delete: borrar el claim puede destruir el volumen; no es un mecanismo de recuperación ni movilidad entre nodos.

Métodos: pg_dump en formato de archivo PostgreSQL, mongodump --archive --gzip y tar de uploads. El conjunto manual 20261025-235337 contiene postgres.dump (31,938 B), mongo.archive.gz (474 B) y uploads.tar.gz (39,043 B), con manifiesto y SHA-256. Cada motor produce su respaldo lógico; no se demuestra un punto transaccional simultáneo entre motores y fotos.

Respaldo diario: tarea Windows TiendaProd-RespaldoDiario a las 20:00, retención de siete copias completas, ZIP cifrado con DPAPI CurrentUser y directorio privado. Respaldo-Programado.ps1 prueba descifrado e integridad antes de retirar temporales; Descifrar-Respaldo.ps1 recupera los tres archivos. La tarea terminó con LastTaskResult=0 en la repetición. Requiere sesión iniciada, Docker y Minikube disponibles. El destino está fuera de los PVC, pero en el mismo equipo: no es copia externa ante pérdida de la laptop.

```
powershell -NoProfile -ExecutionPolicy Bypass -File k8s/minikube/Respaldar-Bases.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File k8s/minikube/Respaldo-Programado.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File k8s/minikube/Programar-Respaldos.ps1 -Hora 20:00
powershell -NoProfile -ExecutionPolicy Bypass -File k8s/minikube/Restaurar-Prueba.ps1 -CarpetaRespaldo C:\laragon\www\sistema\output\respaldos\20261025-235337
```

Restauración: namespace aislado tienda-restore-20261025-235828, tres PVC de 1 GiB y Pods postgres-restore, mongo-restore y fotos-restore. SHA-256 de los tres archivos fue correcto. Se recuperaron 100 prendas, cinco tallas, 500 variantes, dos usuarios, un pedido, un detalle y dos filas outbox; Mongo tenía cero documentos y conservó sus índices. Dos archivos de uploads coincidieron por contenido. El primer intento comparó permisos/GID y falló; se corrigió la validación al hash del contenido y se continuó en el mismo namespace.



Captura 5 · E13 · 25–26/10/2026, entorno aislado. Integridad, conteos e índices recuperados. Los 7.1 s corresponden solo a la continuación, como advierte la propia salida; no son el RTO total.

Después se levantó una aplicación de recuperación con sus propias credenciales y http://127.0.0.1:9081/productos: catálogo, existencia e imagen restaurada visibles. El puerto 9080 estaba ocupado y se cambió a 9081 sin detener el otro servicio. Producción no se sobrescribió. Archivos: k8s/minikube/restauracion/.

Objetivos: RPO 24 h y RTO 30 min. Resultado: no se midió el RTO completo desde el inicio hasta servicio validado; 7.1 s excluye el intento fallido. Tampoco se midió la pérdida temporal real respecto al último respaldo; el RPO no puede garantizarse con la laptop apagada. E13 queda parcial aunque la restauración y su aplicación funcionaron. La propuesta de ampliación es PostgreSQL con replicación/failover, MongoDB replica set y almacenamiento redundante; no se implementó aumentando replicas arbitrariamente.

D10. Pruebas, evidencias y cierre

10.1 Rendimiento y escalado

Se midió lectura autenticada del catálogo con Medir-Catalogo.ps1 desde Windows, por HTTPS y port-forward. Umbral previo de referencia: p95 ≤800 ms y errores <1 %. Cada fase duró cinco minutos; un cliente en la base y 12 en carga. La duración total de la fase incluye arranque/terminación de trabajadores. No se midieron compras, pagos ni 50 usuarios.

Fase	Usuarios	Solicitudes	Fallos	Req/s	p95 (ms)
Base	1	284	0	0.94	73.559
Carga	12	12,514	0	39.25	444.127

Duración real: base 301 s y carga 318.8 s. p95 calculado sobre solicitudes correctas; errores contados aparte. Cumple el umbral para 12 usuarios, no demuestra la meta de 50 ni una comparación causal con/sin HPA.

Ensayo de HPA separado: Job carga-hpa, 24 solicitudes concurrentes y 360 s, obtuvo 29,360 respuestas correctas, cero fallos y 81.56 solicitudes/s promedio. Consultó la sonda de disponibilidad; no se equipara con el benchmark del catálogo. Backend creció de 2 a 4 y 6, luego bajó a 3 y finalmente 2. CPU objetivo 70 % de request 100 mCPU; se ajustó desde la referencia inicial de 60 % para el laboratorio. Frontend quedó en dos. Se observó estabilización, pero no se conserva una serie completa de réplicas Ready/deseadas y CPU/RAM de todas las fases.

```
Solicitudes por segundo (promedio): 81.56
PS C:\Users\josem> minikube -p tienda-prod kubectl -- get hpa -n tienda-prod
NAME       REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
backend    Deployment/backend     cpu: 9%/70%   2         6         6          36m
frontend   Deployment/frontend    cpu: 1%/70%   2         6         2          36m
PS C:\Users\josem> minikube -p tienda-prod kubectl -- delete -f C:\laragon\www\sistema\k8s\minikube\carga-hpa.yaml
networkpolicy.networking.k8s.io "prueba-carga-salida" deleted from tienda-prod namespace
networkpolicy.networking.k8s.io "prueba-carga-entrada" deleted from tienda-prod namespace
job.batch "carga-hpa" deleted from tienda-prod namespace
PS C:\Users\josem> minikube -p tienda-prod kubectl -- describe hpa backend -n tienda-prod
Name:                                                  backend
Namespace:                                             tienda-prod
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Sun, 25 Oct 2026 21:45:31 -0600
Reference:                                             Deployment/backend
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  10% (10m) / 70%
Min replicas:                                          2
Max replicas:                                          6
Deployment pods:                                       3 current / 3 desired
Conditions:
  Type            Status  Reason               Message
  ----            ------  ------               -------
  AbleToScale     True    ScaleDownStabilized  recent recommendations were higher than current one, applying the highest recent recomme
  ScalingActive   True    ValidMetricFound     the HPA was able to successfully calculate a replica count from cpu resource utilizatio
st)
  ScalingLimited  False   DesiredWithinRange   the desired count is within the acceptable range
  ScaledToZero    False   NotScaledToZero      the HPA controller did not scale the workload to zero
Events:
  Type    Reason             Age   From                       Message
  ----    ------             ----  ----                       -------
  Normal  SuccessfulRescale  11m   horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization (percentage of request) abo
  Normal  SuccessfulRescale  10m   horizontal-pod-autoscaler  New size: 6; reason: cpu resource utilization (percentage of request) abo
  Normal  SuccessfulRescale  18s   horizontal-pod-autoscaler  New size: 3; reason: All metrics below target
PS C:\Users\josem> minikube -p tienda-prod kubectl -- get hpa -n tienda-prod
NAME       REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend    Deployment/backend     cpu: 10%/70%   2         6         3          37m
frontend   Deployment/frontend    cpu: 1%/70%    2         6         2          37m
PS C:\Users\josem>
```

Captura 6 · E06 · 25/10/2026, producción. Eventos SuccessfulRescale de 4, 6 y 3 réplicas y condición ScaleDownStabilized. El retorno final a dos se comprobó después; no se modifica esta captura para simularlo.

Balanceo: Service selecciona varios endpoints y la consulta del corte ubica frontend/backend en m02 y m03. Esa distribución y la continuidad observada no prueban por sí solas que cada réplica atendió solicitudes: falta una correlación explícita por Pod o un identificador temporal. E07 se marca parcial. Desarrollo sí se escaló manualmente 1→2→1, con rollout correcto.

10.2 Matriz de aceptación y trazabilidad

Cada fila enlaza requisito, ambiente, acción y resultado con los archivos del anexo. Aprobada significa que existe evidencia concreta de ese alcance; parcial no se convierte en aprobación por una captura aislada.

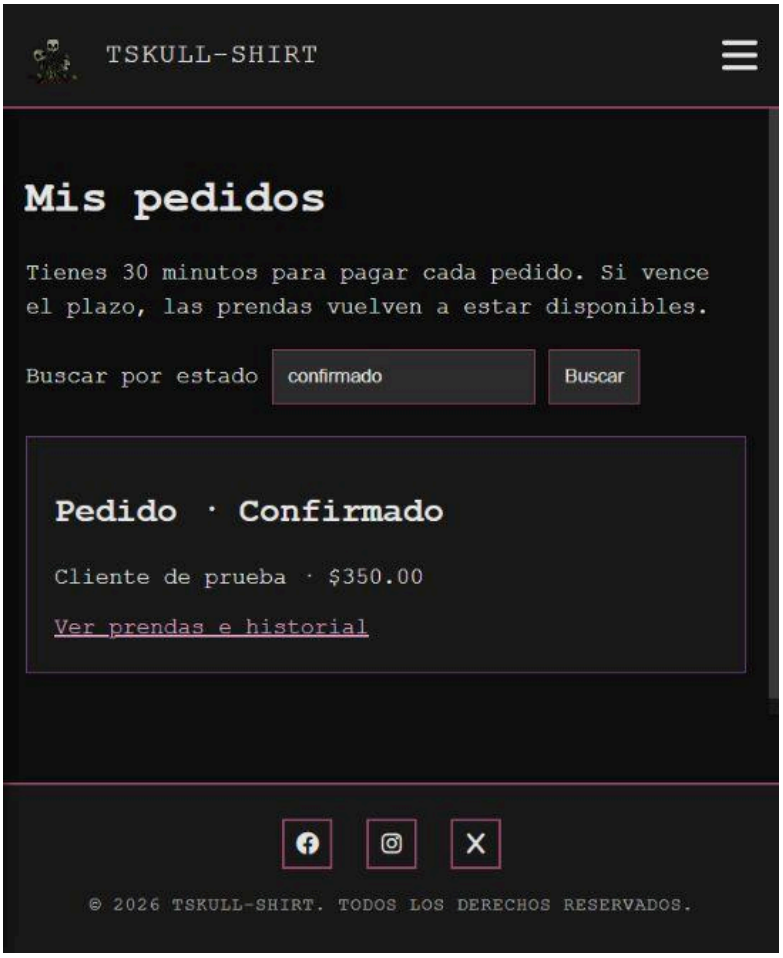
E01 Aprobada: perfiles independientes; dev un nodo y prod tres Ready. E02 Aprobada: HTTPS distintos y datos/PVC separados; acceso LAN a prod y rechazo de dev confirmado por el usuario el 31/10/2026, sin captura nueva del teléfono adjunta.

E03 Parcial: flujo de pedidos, roles y edición limitada probados; falta expediente completo de CRUD de tres entidades por interfaz/API y sesión expirada. E04 Aprobada para uso funcional: catálogo PostgreSQL e historial MongoDB visible, con outbox y sincronización real. E05 Parcial: persistencia y restauración comprobadas; falta registro homogéneo antes/después de recrear cada Pod de datos.

E06 Parcial: crecimiento 2→4→6 y descenso a 3; el mínimo de dos se comprobó después, sin una serie completa exportada de la misma ventana. E07 Parcial: réplicas entre workers, sin trazabilidad de solicitudes por réplica. E08 Parcial: backend permitido, frontend bloqueado y DNS sano; falta prueba separada completa de ingress/egress. E09 Parcial: rol de aplicación, 12 verificaciones RBAC y usuarios limitados; falta prueba negativa documentada de todos los privilegios de base.

E10 Parcial: métricas de ambos motores y dos alertas activadas/resueltas; faltan centralización de logs y cobertura operativa completa. E11 Parcial: eliminación de un Pod y drain selectivo medidos, sin ensayo completo de pérdida de worker. E12 Aprobada: actualización/rollback con imágenes y HTTPS verificados. E13 Parcial: datos, índices, archivos y aplicación restaurados; RPO/RTO completos no medidos. E14 Pendiente: no se ejecutó reconstrucción completa en recursos limpios; tampoco hay tag final de entrega.

Registros: funcionales/agente-20261027/RESULTADOS.md y SQL final; rendimiento/20261026-211312/resumen.csv y solicitudes; seguridad/rbac-20261026-002647.csv y exportaciones; tls-postgres/comprobacion-20261026-010203.txt; restauracion/20261025-235828/resultado.json; imágenes/*-cves.txt. Las rutas completas, fechas, comandos y resultados están en el registro de anexos. No se adjuntan secretos ni respaldos con credenciales.



Captura 7 · E02/E03 · 27/10/2026, producción. Vista real de pedidos del cliente con búsqueda por estado y sin número global. La prueba posterior del teléfono fue reportada por el usuario, no es esta captura.

10.3 Resultados y conclusiones

La tienda funciona sobre dos clústeres independientes, conserva datos y archivos, valida reservas por talla y usa las dos bases para funciones visibles. La capa de aplicación dispone de réplicas, HPA y recuperación; los ensayos muestran resultados y límites concretos, incluido un 502 transitorio y una restauración reanudada. El acceso del celular a producción y el rechazo de desarrollo se confirmaron al cierre.

El aprendizaje principal fue separar persistencia, disponibilidad y respaldo: un PVC no replica un motor, dos Pods no eliminan el fallo del host y una copia cifrada no demuestra recuperación hasta restaurarla. La mayor prioridad de cierre es completar E14 y la trazabilidad de entrega; después, medir RPO/RTO, ampliar observabilidad, evidencias por réplica y pruebas de seguridad/CRUD. No asigno una calificación ni presento el expediente como cumplimiento total: la evaluación corresponde al docente.

10.4 Referencias oficiales y anexos

Fuente de requisitos: Practica_Individual_Kubernetes_Desarrollo_Produccion-2.pdf, 21 páginas; en especial §§4–12 y D01–D10. Consulta del informe: 31/10/2026. Las referencias técnicas siguientes fundamentan D02, D06, D07 y D09; las versiones implementadas se fijan en D05 y no se sustituyen por las últimas publicadas.

Minikube, Basic controls: https://minikube.sigs.k8s.io/docs/handbook/controls/

Kubernetes 1.37, Horizontal Pod Autoscaling: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Kubernetes 1.37, Network Policies: https://kubernetes.io/docs/concepts/services-networking/network-policies/

Traefik, TLSStore: https://doc.traefik.io/traefik/reference/routing-configuration/kubernetes/crd/tls/tlsstore/

PostgreSQL 17, SQL Dump: https://www.postgresql.org/docs/17/backup-dump.html

MongoDB, Atomicity and Transactions: https://www.mongodb.com/docs/manual/core/write-operations-atomicity/

Anexos técnicos: Descargar anexos editables y registros. paquete con diagramas SVG/PNG, diccionarios de datos, manual ampliado, registro E01–E14, resultados de carga y capturas seleccionadas. Material de capturas originales: https://docs.google.com/document/d/1XP_CFyjxby2UubWK8LbBRpLdAKPgJoBxf4kUjAylaFA/edit. El código final debe cerrarse con commit/tag y revisión de exclusión de secretos antes de entregar.